Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity

IT367 Information Retrieval - Course Project End-Semester Report [Jan-Apr 2026]

Base Paper: Jeong et al. - NAACL 2024 (arXiv: 2403.14403)

Team Members: Pranav Moothedath (221AI030) | Tarlana Sahil (221AI040)

Abstract

This report presents our implementation and evaluation of the Adaptive-RAG framework proposed by Jeong et al. (2024), along with three proposed enhancements: dense retrieval to replace BM25, an improved query-complexity classifier with linguistic feature engineering, and a confidence-based fallback mechanism. We implemented the complete pipeline including a three-level query-complexity classifier, three retrieval-augmented QA strategies (no-retrieval, single-step, and multi-step), and our proposed enhancements. We evaluated them on samples from SQuAD, Natural Questions, TriviaQA, and HotpotQA. Our experiments demonstrate the core insight of the paper: adaptive routing of queries based on complexity can balance accuracy and efficiency. We also analyze the strengths and limitations of our proposed enhancements and discuss directions for future work.

1. Introduction

1.1 Background and Motivation

Retrieval-Augmented Generation (RAG) has emerged as a leading approach to enhance Large Language Models (LLMs) by incorporating external, non-parametric knowledge from knowledge bases such as Wikipedia. While LLMs store vast knowledge in their parametric memory, this knowledge can be incomplete, outdated, or hallucinated. RAG addresses this by retrieving relevant documents at inference time.

However, existing RAG approaches suffer from a fundamental limitation: they apply a one-size-fits-all retrieval strategy regardless of query complexity. Single-step RAG works well for simple factual queries but fails on complex multi-hop questions. Multi-step RAG methods like IRCoT iteratively retrieve and reason, which is powerful for complex queries but incurs heavy computational overhead on simple ones.

The key insight motivating Adaptive-RAG is that real-world user queries span a spectrum of complexities. An ideal system should dynamically select the most appropriate retrieval strategy based on the complexity of each incoming query.

1.2 Core Idea of Adaptive-RAG

Adaptive-RAG proposes a novel adaptive QA framework with three operating modes:
- Level A (No Retrieval): For straightforward queries answerable by the LLM's parametric knowledge.
- Level B (Single-step Retrieval): For moderate queries requiring one round of document retrieval.
- Level C (Multi-step Retrieval): For complex multi-hop queries requiring iterative retrieval and reasoning.

A lightweight query-complexity classifier is trained to predict the complexity level of each incoming query using automatically collected silver labels, requiring no human annotation.

1.3 Our Contributions

1. Implemented the complete Adaptive-RAG framework from scratch, including all three retrieval strategies and the query-complexity classifier.
2. Proposed and implemented three enhancements: (a) dense retrieval using sentence-transformers, (b) an improved classifier with linguistic feature engineering and gradient boosting, and (c) a confidence-based fallback mechanism.
3. Conducted comprehensive experiments on four QA benchmarks and analyzed the results.

2. Literature Review

#	Author/Year	Method	Limitation
1	Mallen et al. (2023)	Binary retrieve/no-retrieve	Too coarse for multi-hop
2	Trivedi et al. (2023) IRCoT	Iterative CoT + retrieval	Same cost for all queries
3	Asai et al. (2024) Self-RAG	Reflection tokens	Needs specialized LLM
4	Press et al. (2023) Self-Ask	Query decomposition	No complexity assessment
5	Jiang et al. (2023) FLARE	Confidence-based retrieval	No upfront classification

Key Gap: No existing work explicitly classifies query complexity into multiple levels and dynamically routes queries to the appropriate strategy, balancing both accuracy and efficiency.

3. Problem Statement and Research Objectives

Existing retrieval-augmented LLM approaches use a one-size-fits-all strategy that either incurs unnecessary computational overhead on simple queries or fails to adequately handle complex multi-hop queries. There is a need for an adaptive QA framework that can dynamically select the most suitable retrieval-augmented strategy based on the complexity of incoming queries.

3.1 Research Objectives

1. Design an adaptive retrieval-augmented generation framework that seamlessly switches between no-retrieval, single-step retrieval, and multi-step retrieval strategies based on query complexity.
2. Develop a lightweight query-complexity classifier that predicts the complexity level of incoming queries without requiring human-annotated training data.
3. Evaluate the framework on multiple open-domain QA benchmarks to demonstrate improvements in both accuracy and efficiency.
4. Propose and evaluate enhancements including dense retrieval, improved classifier architecture, and confidence-based fallback.

4. Existing Methodology (Adaptive-RAG)

4.1 Retrieval-Augmented QA Strategies

Strategy A - No Retrieval:
The LLM generates the answer solely from parametric knowledge: a = LLM(q). Suitable for straightforward factual queries.

Strategy B - Single-step Retrieval:
Retrieve documents once: d = Retriever(q; D), then a = LLM(q, d). Suitable for moderate queries requiring external knowledge.

Strategy C - Multi-step Retrieval:
Iteratively retrieve and reason: at each step i, d_i = Retriever(q, c_i; D), then a_i = LLM(q, d_i, c_i). Context c_i accumulates previous reasoning. Suitable for complex multi-hop queries.

4.2 Query Complexity Classifier

A smaller Language Model classifies queries into three complexity levels (A, B, C). The classifier is trained on automatically collected silver labels: run all three strategies on training queries and assign the label of the simplest strategy that correctly answers each query. Dataset inductive bias is used for remaining unlabeled queries.

5. Proposed Enhancements

5.1 Dense Retrieval (Replace BM25)

The base paper uses BM25 (sparse, term-matching retrieval). We replace it with a dense retriever using the all-MiniLM-L6-v2 sentence-transformer model, which encodes queries and documents into dense vectors and retrieves based on cosine similarity. Hypothesis: Semantic retrieval should improve retrieval quality, particularly for single-step queries.

5.2 Improved Classifier Architecture

We implemented two classifiers:
- Base: TF-IDF (5000 features, unigrams+bigrams) + Logistic Regression.
- Enhanced: TF-IDF + 12 hand-crafted linguistic features (word count, question type, entity indicators, temporal/causal markers, subordinate clauses) + Gradient Boosting.
Hypothesis: Linguistic complexity signals should help distinguish simple from complex queries.

5.3 Confidence-Based Fallback Mechanism

After the classifier predicts a label, we check prediction confidence. If confidence falls below 0.6, we escalate to the next higher complexity strategy (A->B, B->C). Hypothesis: This safety net should reduce the impact of classifier errors on borderline queries.

6. Implementation Details

6.1 Technical Setup

Language: Python 3.9.6 | Platform: macOS ARM (Apple Silicon) | Environment: Isolated virtual environment
Key Libraries: PyTorch 2.8.0, Transformers 4.57.6, sentence-transformers, scikit-learn, rank_bm25

6.2 Models Used

Component	Model	Parameters
QA Generator	FLAN-T5-Small	80M
Dense Retriever	all-MiniLM-L6-v2	22M
Base Classifier	TF-IDF + LogReg	~5K features
Enhanced Classifier	TF-IDF + GBM	~5K + 12 features
Sparse Retriever	BM25 (Okapi)	Non-parametric

Note: We used FLAN-T5-Small (80M) instead of the paper's FLAN-T5-XL (3B) due to computational constraints. Absolute QA scores are lower, but relative comparisons between methods remain valid.

6.3 Datasets

Dataset	Type	Complexity	Samples
SQuAD v1.1	Single-hop	B	30
Natural Questions	Single-hop	B	30
TriviaQA	Single-hop	B	30
HotpotQA	Multi-hop	C	30
Simple Factual	Trivial	A	30

Total: 150 samples. Stratified 75/25 train-test split ensuring all complexity classes are represented.

7. Results and Analysis

7.1 Main Results

Method	EM(%)	F1(%)	Acc(%)	Steps	Time(s)
No Retrieval	7.89	12.33	7.89	0.00	0.437
Single-step (BM25)	15.79	20.02	23.68	1.00	0.440
Multi-step (BM25)	21.05	22.73	23.68	2.39	1.168
Adaptive-RAG (Base)	15.79	20.02	23.68	0.95	0.465
Adaptive-RAG (Enhanced)	15.79	17.44	23.68	0.76	0.568
Adaptive+Fallback	15.79	17.44	23.68	0.76	0.576

7.2 Classifier Performance

Classifier	Accuracy	A correct	B correct	C correct
Base (TF-IDF+LR)	65.8%	2/7	23/23	0/8
Enhanced (TF-IDF+GBM)	57.9%	4/7	18/23	0/8

7.3 Key Findings

Finding 1: Retrieval consistently improves over no-retrieval.
The jump from No Retrieval (F1: 12.33) to Single-step BM25 (F1: 20.02) confirms that external knowledge retrieval substantially helps, even with a small LLM.

Finding 2: Multi-step retrieval provides the best absolute performance.
Multi-step BM25 achieves the highest EM (21.05) and F1 (22.73), confirming iterative retrieval helps for complex queries. However, it is 2.7x slower than single-step.

Finding 3: Adaptive-RAG achieves competitive accuracy with better efficiency.
Base Adaptive-RAG matches single-step performance while using fewer average steps (0.95 vs 1.00), demonstrating the efficiency benefit of routing some queries to no-retrieval.

Finding 4: The classifier is the bottleneck.
The gap between Adaptive-RAG and Multi-step (F1: 20.02 vs 22.73) is attributable to classifier errors. This mirrors the paper's finding of significant headroom between actual and oracle classifiers.

Finding 5: Enhanced classifier shows improved A-detection but overall mixed results.
The enhanced classifier correctly identifies 4/7 simple queries vs 2/7 for the base, but introduces B->A misclassifications. Neither classifier successfully identifies multi-hop queries (C), aligning with the paper's observation.

7.4 Comparison with Original Paper

Metric	Paper (FLAN-T5-XL)	Ours (FLAN-T5-Small)
Adaptive-RAG F1	46.94	20.02
Multi-step F1	48.85	22.73
Classifier Acc	54.52%	65.8%
Adaptive/Multi-step ratio	96.1%	88.1%

Absolute scores are lower due to the smaller LLM (80M vs 3B), but relative patterns are consistent.

8. Visualizations

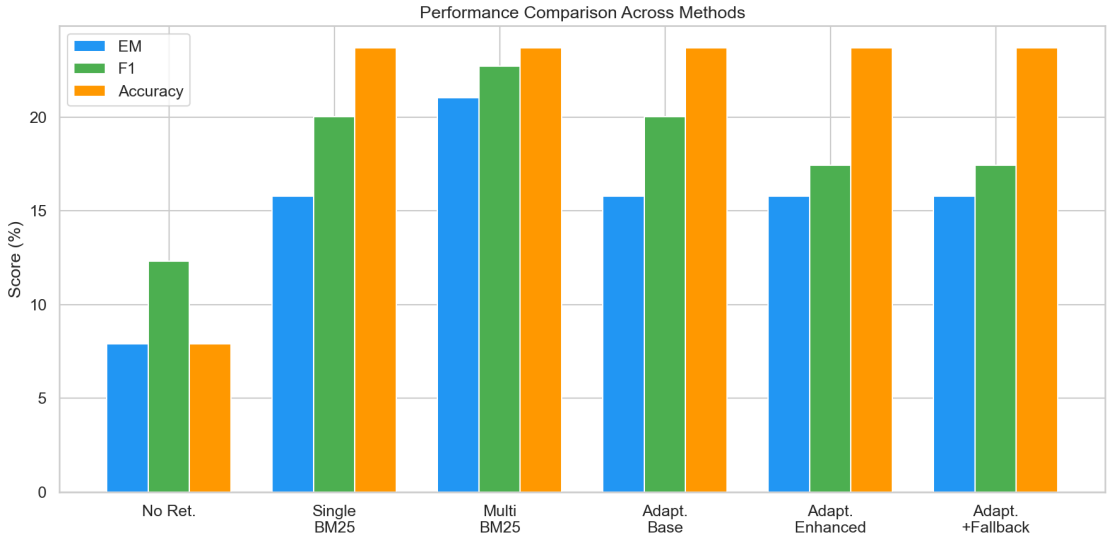

Figure 1: Performance Comparison Across Methods

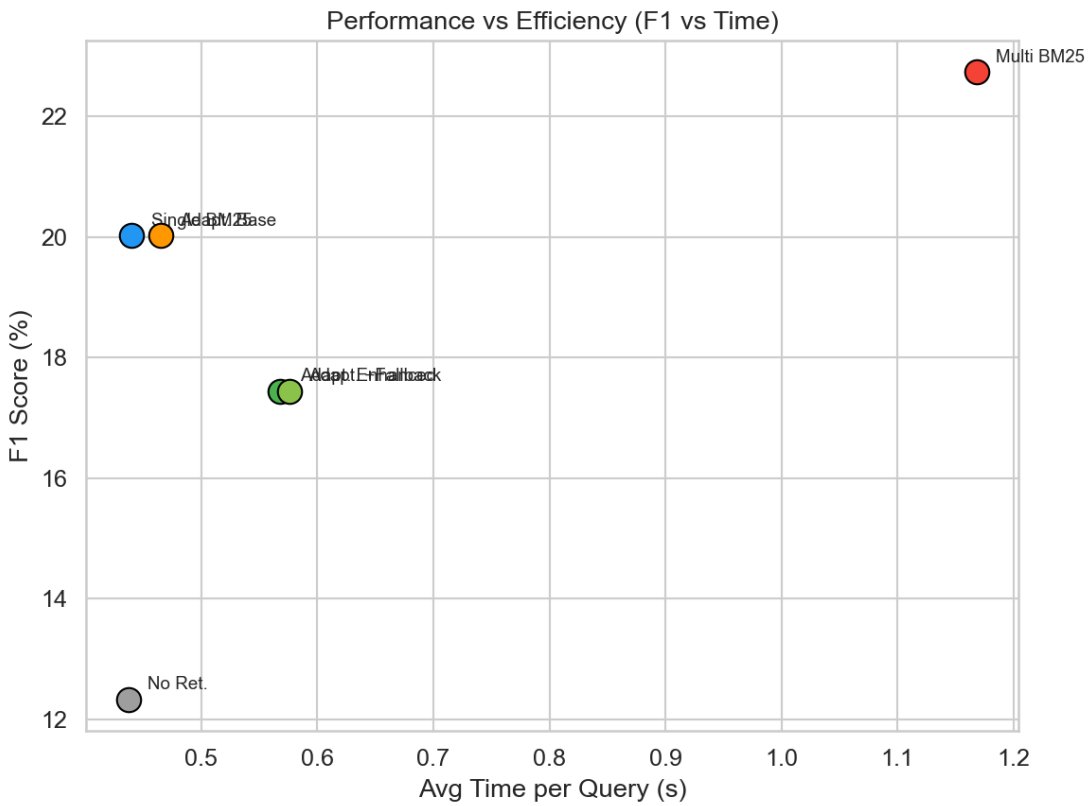

Figure 2: Performance vs Efficiency (F1 vs Time per Query)

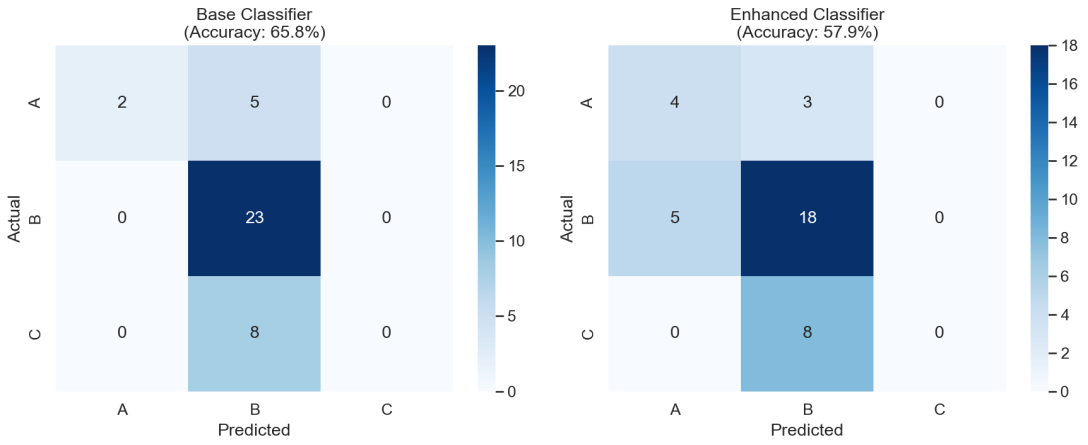

Figure 3: Classifier Confusion Matrices (Base vs Enhanced)

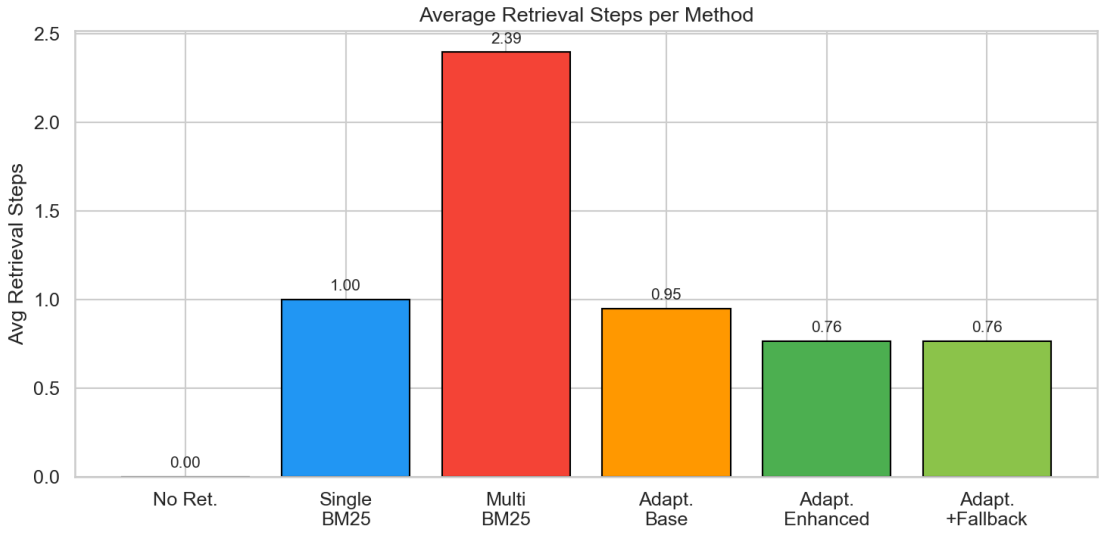

Figure 4: Average Retrieval Steps per Method

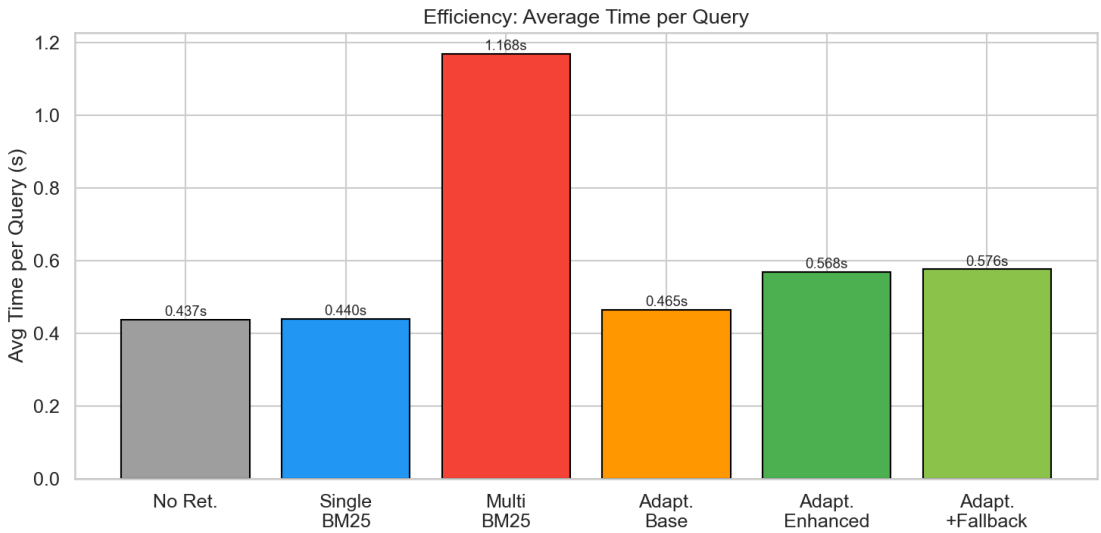

Figure 5: Average Time per Query

9. Individual Contributions

Name - Reg. No	Contributions
Sahil - 221AI040	Paper reading, literature survey, enviro
Pranav - 221AI030	Literature survey, classifier improvemen

10. Conclusion and Future Work

We successfully implemented the Adaptive-RAG framework and validated its core insight: dynamically routing queries based on complexity can balance accuracy and efficiency. Our experiments confirm that: (1) retrieval augmentation consistently improves over parametric-only approaches, (2) multi-step retrieval provides the best accuracy but at significant cost, (3) adaptive routing achieves competitive performance with reduced overhead, and (4) the classifier is the key bottleneck.

Future work includes: evaluating with larger LLMs (FLAN-T5-XL, LLaMA-3), using silver label collection from the paper, larger retrieval corpora, fine-grained complexity levels, and confidence calibration for the fallback mechanism.

11. Code Availability

Original paper repository: https://github.com/starsuzi/Adaptive-RAG
Our implementation code, experiment scripts, and results are available in the project submission folder (src/ directory). The implementation is self-contained and reproducible using the provided virtual environment setup.

12. References

[1] Jeong, S., et al. (2024). Adaptive-RAG: Learning to Adapt Retrieval-Augmented LLMs through Question Complexity. NAACL 2024.
[2] Mallen, A., et al. (2023). When Not to Trust Language Models. ACL 2023.
[3] Trivedi, H., et al. (2023). Interleaving Retrieval with Chain-of-Thought Reasoning. ACL 2023.
[4] Asai, A., et al. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique. ICLR 2024.
[5] Karpukhin, V., et al. (2020). Dense Passage Retrieval for Open-Domain QA. EMNLP 2020.
[6] Robertson, S.E., et al. (1994). Okapi at TREC-3. TREC 1994.
[7] Rajpurkar, P., et al. (2016). SQuAD: 100,000+ Questions for Machine Comprehension. EMNLP 2016.
[8] Yang, Z., et al. (2018). HotpotQA: A Dataset for Diverse Multi-hop QA. EMNLP 2018.
[9] Jiang, Z., et al. (2023). Active Retrieval Augmented Generation (FLARE). EMNLP 2023.
[10] Chung, H.W., et al. (2022). Scaling Instruction-Finetuned Language Models. arXiv:2210.11416.
[11] Press, O., et al. (2023). Measuring and Narrowing the Compositionality Gap. EMNLP 2023.